Aarya Arban
T11 - 05
Assignment No. 8

Aim: Write a program to implement k - means clustering.

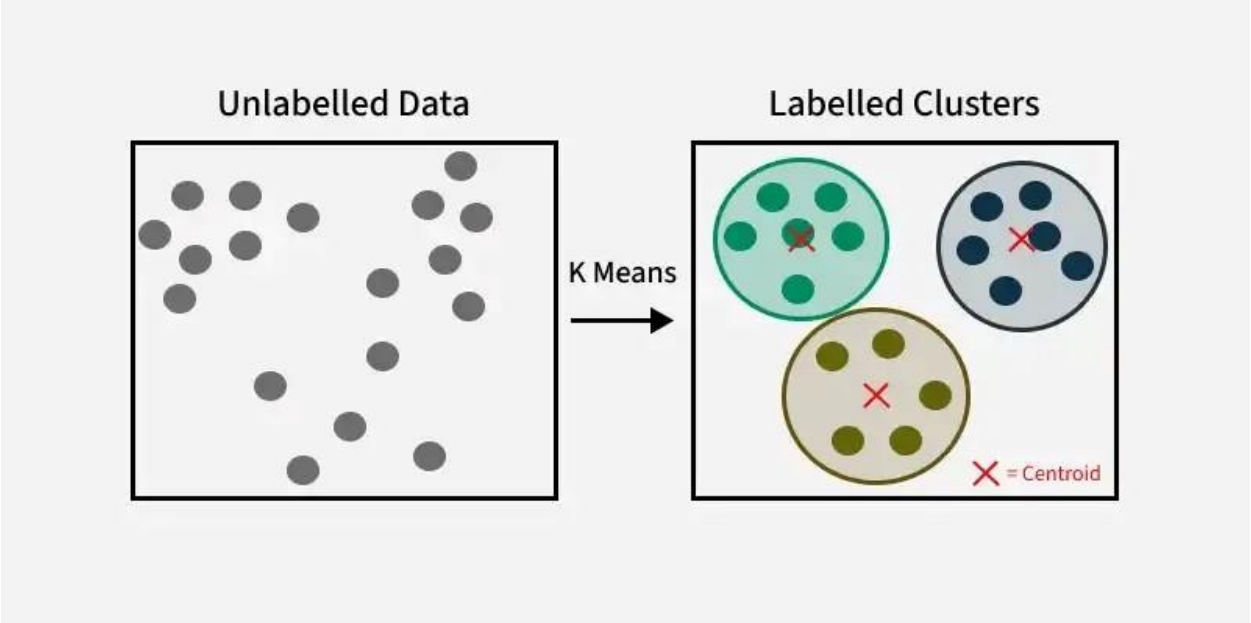
Theory:

K-means clustering is a technique used to organize data into groups based on their similarity. For example online store uses K-Means to group customers based on purchase frequency and spending creating segments like Budget Shoppers, Frequent Buyers and Big Spenders for personalised marketing.

The algorithm works by first randomly picking some central points called centroids and each data point is then assigned to the closest centroid forming a cluster. After all the points are assigned to a cluster the centroids are updated by finding the average position of the points in each cluster. This process repeats until the centroids stop changing forming clusters. The goal of clustering is to divide the data points into clusters so that similar data points belong to same group.

How does k-means clustering work?

We are given a data set of items with certain features and values for these features (like a vector). The task is to categorize those items into groups. To achieve this, we will use the K-means algorithm. ‘K’ in the name of the algorithm represents the number of groups/clusters we want to classify our items into.



The algorithm will categorize the items into k groups or clusters of similarity. To calculate that similarity, we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. First, we randomly initialize k points, called means or cluster centroids.
2. We categorize each item to its closest mean, and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.
3. We repeat the process for a given number of iterations and at the end, we have our clusters.

The "points" mentioned above are called means because they are the mean values of the items categorized in them. To initialize these means, we have a lot of options. An intuitive method is to initialize the means at random items in the data set. Another method is to initialize the means at random values between the boundaries of the data set. For example for a feature x the items have values in [0,3] we will initialize the means with values for x at [0,3].

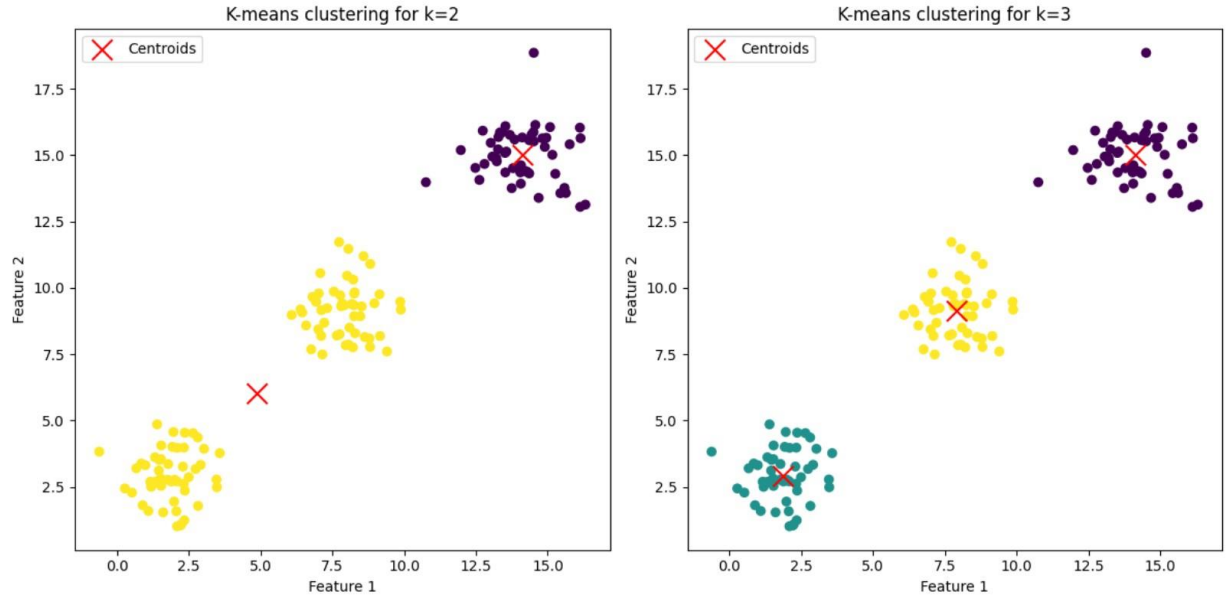
Code:

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Example of multidimensional data with 5 features (replace with your actual data)
# Each row represents a data point, and there are 5 features per data point.
data = np.array([
    [1.0, 2.0, 3.0, 4.0, 5.0, 10.0, 11.0, 12.0, 13.0, 14.0],
    [2.0, 3.0, 4.0, 5.0, 6.0, 12.0, 13.0, 14.0, 15.0, 16.0],
    [3.0, 4.0, 5.0, 6.0, 7.0, 12.0, 13.0, 14.0, 15.0, 16.0],
    [10.0, 11.0, 12.0, 13.0, 14.0, 1.0, 2.0, 3.0, 4.0, 5.0],
    [11.0, 12.0, 13.0, 14.0, 15.0, 1.0, 2.0, 3.0, 4.0, 5.0],
    [12.0, 13.0, 14.0, 15.0, 16.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [18.0, 19.0, 20.0, 21.0, 22.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [19.0, 20.0, 21.0, 22.0, 23.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [20.0, 21.0, 22.0, 23.0, 24.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [30.0, 31.0, 32.0, 33.0, 34.0, 2.0, 3.0, 4.0, 5.0, 6.0]
])

# Function to perform k-means clustering and return the error (inertia)
def kmeans_clustering(data, k):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    return kmeans.inertia_  # Return the error value (inertia)

# Perform k-means clustering for k=2 and k=3
error_k2 = kmeans_clustering(data, 2)
error_k3 = kmeans_clustering(data, 3)

# Print the error values for both k=2 and k=3
print(f"Error for k=2: {error_k2}")
print(f"Error for k=3: {error_k3}")
```

```
[ ]  import numpy as np
     from sklearn.cluster import KMeans
     import matplotlib.pyplot as plt

     # Generate random two-dimensional data with 150 data points
     # Using normal distribution to create scattered data points
     np.random.seed(42)  # For reproducibility
     data = np.vstack([
         np.random.normal(loc=[2, 3], scale=[1, 1], size=(50, 2)),  # Cluster 1
         np.random.normal(loc=[8, 9], scale=[1, 1], size=(50, 2)),  # Cluster 2
         np.random.normal(loc=[14, 15], scale=[1, 1], size=(50, 2))  # Cluster 3
     ])

     # Function to perform k-means clustering and return the error (inertia)
     def kmeans_clustering(data, k):
         kmeans = KMeans(n_clusters=k)
         kmeans.fit(data)
         return kmeans.inertia_  # Return the error value (inertia)

     # Perform k-means clustering for k=2 and k=3
     error_k2 = kmeans_clustering(data, 2)
     error_k3 = kmeans_clustering(data, 3)

     # Print the error values for both k=2 and k=3
     print(f"Error for k=2: {error_k2}")
     print(f"Error for k=3: {error_k3}")

     # Compare the errors to find the ideal k
     if error_k2 < error_k3:
         print("k=2 has the lower error, so it is the ideal choice.")
     else:
         print("k=3 has the lower error, so it is the ideal choice.")
```

```python
# Optional: Plot the data and cluster centers for both k=2 and k=3
kmeans_2 = KMeans(n_clusters=2).fit(data)
kmeans_3 = KMeans(n_clusters=3).fit(data)

plt.figure(figsize=(12, 6))

# Plot for k=2
plt.subplot(1, 2, 1)
plt.scatter(data[:, 0], data[:, 1], c=kmeans_2.labels_, cmap='viridis')
plt.title('K-means clustering for k=2')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(kmeans_2.cluster_centers_[:, 0], kmeans_2.cluster_centers_[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

# Plot for k=3
plt.subplot(1, 2, 2)
plt.scatter(data[:, 0], data[:, 1], c=kmeans_3.labels_, cmap='viridis')
plt.title('K-means clustering for k=3')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(kmeans_3.cluster_centers_[:, 0], kmeans_3.cluster_centers_[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

plt.tight_layout()
plt.show()
```

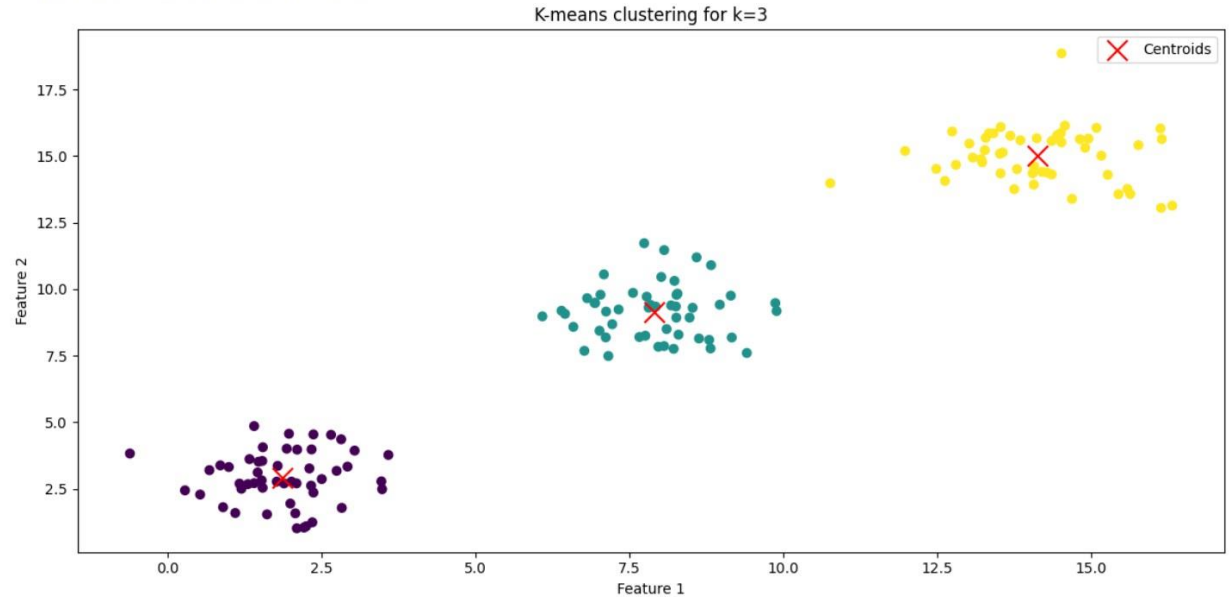
```
Error for k=2: 2113.4826231862244
Error for k=3: 286.1842142470732
k=3 has the lower error, so it is the ideal choice.
```



```python
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate random two-dimensional data with 150 data points
# Using normal distribution to create scattered data points
np.random.seed(42)  # For reproducibility
data = np.vstack([
    np.random.normal(loc=[2, 3], scale=[1, 1], size=(50, 2)),  # Cluster 1
    np.random.normal(loc=[8, 9], scale=[1, 1], size=(50, 2)),  # Cluster 2
    np.random.normal(loc=[14, 15], scale=[1, 1], size=(50, 2))  # Cluster 3
])

# Function to perform k-means clustering and return the error (inertia)
def kmeans_clustering(data, k):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    return kmeans

# Perform k-means clustering for k=3
kmeans_3 = kmeans_clustering(data, 3)

# Get the centroids of the clusters
centroids = kmeans_3.cluster_centers_

# Calculate Euclidean distance between centroids
def euclidean_distance(p1, p2):
    return np.sqrt(np.sum((p2 - p1) ** 2))
```

```python
# Calculate the distance between all pairs of centroids
distances = {}
for i in range(len(centroids)):
    for j in range(i + 1, len(centroids)):
        distance = euclidean_distance(centroids[i], centroids[j])
        distances[f"Centroid {i+1} to Centroid {j+1}"] = distance

# Print distances between the centroids
for pair, distance in distances.items():
    print(f"Distance between {pair}: {distance:.2f}")

# Optional: Plot the data and cluster centers for k=3
plt.figure(figsize=(12, 6))

# Plot for k=3
plt.scatter(data[:, 0], data[:, 1], c=kmeans_3.labels_, cmap='viridis')
plt.title('K-means clustering for k=3')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(centroids[:, 0], centroids[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

plt.tight_layout()
plt.show()
```

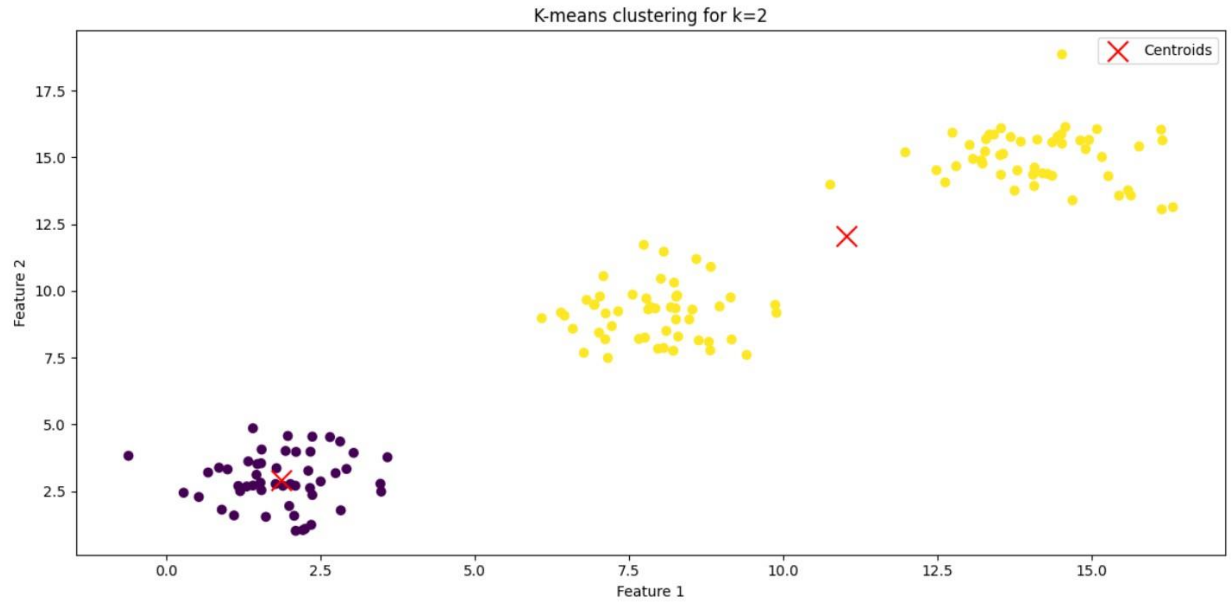
```
Distance between Centroid 1 to Centroid 2: 8.66
Distance between Centroid 1 to Centroid 3: 17.21
Distance between Centroid 2 to Centroid 3: 8.55
```



```python
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate random two-dimensional data with 150 data points
# Using normal distribution to create scattered data points
np.random.seed(42)  # For reproducibility
data = np.vstack([
    np.random.normal(loc=[2, 3], scale=[1, 1], size=(50, 2)),  # Cluster 1
    np.random.normal(loc=[8, 9], scale=[1, 1], size=(50, 2)),  # Cluster 2
    np.random.normal(loc=[14, 15], scale=[1, 1], size=(50, 2))  # Cluster 3
])

# Function to perform k-means clustering and return the error (inertia)
def kmeans_clustering(data, k):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    return kmeans

# Perform k-means clustering for k=2
kmeans_2 = kmeans_clustering(data, 2)

# Get the centroids of the clusters
centroids_2 = kmeans_2.cluster_centers_

# Calculate Euclidean distance between centroids
def euclidean_distance(p1, p2):
    return np.sqrt(np.sum((p2 - p1) ** 2))
```

```python
# Calculate the distance between the centroids for k=2
distances_2 = {}
for i in range(len(centroids_2)):
    for j in range(i + 1, len(centroids_2)):
        distance = euclidean_distance(centroids_2[i], centroids_2[j])
        distances_2[f"Centroid {i+1} to Centroid {j+1}"] = distance

# Print distances between the centroids for k=2
for pair, distance in distances_2.items():
    print(f"Distance between {pair}: {distance:.2f}")

# Optional: Plot the data and cluster centers for k=2
plt.figure(figsize=(12, 6))

# Plot for k=2
plt.scatter(data[:, 0], data[:, 1], c=kmeans_2.labels_, cmap='viridis')
plt.title('K-means clustering for k=2')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(centroids_2[:, 0], centroids_2[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

plt.tight_layout()
plt.show()
```

```
Distance between Centroid 1 to Centroid 2: 12.94
```



Conclusion:

The K-Means clustering algorithm successfully grouped the data, but choosing **k=2** for a dataset with three natural clusters led to suboptimal results. The computed centroids and their distances indicate how the algorithm merged clusters. Visualization shows the grouping, but selecting the right **k** using methods like the **Elbow Method** or **Silhouette Score** is crucial for better accuracy.